Product Note: Card Spend AI

Failure Modes

1. **Unknown merchants** — the biggest gap. Any merchant not in the database falls to keyword or hint inference, producing low-confidence recommendations.
2. **Noisy merchant strings** — credit card statements produce strings like "GRAB* A-4KXYZW". Preprocessing catches common patterns but new formats break silently.
3. **Ambiguous category hints** — "travel" could mean airlines, hotels, or a travel portal, each with different reward rates. We check all sub-categories and pick the best reward, but the user doesn't know we made that choice.
4. **Close card calls** — when two cards are within 20% reward rate, the "best" card depends on factors we don't model: spending caps, sign-up bonuses, monthly minimums.

What to Log

Every recommendation should store: raw merchant string, normalized string, resolved category, match type and confidence, fuzzy score (when applicable), category hint vs. category resolved, all cards ranked with rates, and whether `update_needed` was triggered. This enables two feedback loops: improving merchant coverage and improving recommendation quality.

What to measure from these logs: resolution quality (percentage of queries hitting exact match vs. fuzzy vs. keyword vs. hint fallback — goal is shifting toward exact over time), inference accuracy (approval rate on the review queue — 80%+ means the guessing logic works), recommendation adoption (did the user actually use the suggested card), and coverage growth rate (new merchants added per week through the learning loop).

How to Improve Coverage

Build a merchant-learning loop. Every low-confidence or missing-merchant case gets logged, reviewed, and promoted into the merchant database. This is the highest-leverage investment because:

- The system's main failure mode is merchant coverage
- Better merchant resolution improves every downstream recommendation immediately
- It turns uncertain runtime guesses into permanent knowledge-base improvements
- It's much more practical than building a smarter model from scratch

The workflow: system flags uncertain cases → reviewer approves/corrects → merchant and aliases added to database → future queries hit exact/fuzzy match instead of inference.

2-Week Roadmap

Week 1: Merchant learning pipeline — Build a review UI for the update queue (approve/reject/correct). Add approved merchants and aliases back into the database automatically. Track approval rate to measure inference quality.

Week 2: Recommendation quality — Add user feedback capture (did you use this card?). Replace keyword matching with sentence-transformers for better unknown merchant categorization. Add spending cap awareness to card ranking.

The merchant learning pipeline comes first because it compounds: every merchant added improves all future recommendations with zero model changes.